

# Informe de Implementación

## Despliegue Continuo Robusto con un Pipeline de CI/CD

### usando GitHub Actions

Juan Sebastian Otero Vega - 2220053

Farid Camilo Rojas Vargas - 2220051

Juan David Pallares Pallares - 2220079

Mayo 2026

Link del Código:

[https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse\\_vers2.git](https://github.com/JuanSe2731/Proyecto-Spring-Boot-E-converse_vers2.git)

# Índice

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                  | <b>3</b>  |
| <b>2. Descripción general del sistema</b>               | <b>3</b>  |
| 2.1. Stack tecnológico . . . . .                        | 3         |
| 2.2. Arquitectura de la aplicación . . . . .            | 3         |
| <b>3. Estructura del repositorio</b>                    | <b>4</b>  |
| <b>4. Conceptos fundamentales de CI/CD</b>              | <b>5</b>  |
| 4.1. Integración Continua (CI) . . . . .                | 5         |
| 4.2. Entrega Continua (CD) . . . . .                    | 5         |
| 4.3. Conceptos de GitHub Actions . . . . .              | 6         |
| <b>5. Estrategia de Branching</b>                       | <b>6</b>  |
| <b>6. Explicación del Pipeline CI/CD</b>                | <b>7</b>  |
| 6.1. Job 1: Build & Test . . . . .                      | 7         |
| 6.2. Job 2: Docker Build & Push . . . . .               | 9         |
| 6.3. Job 3: Deploy . . . . .                            | 11        |
| 6.4. Job 4: Notificaciones . . . . .                    | 13        |
| <b>7. Pruebas unitarias implementadas</b>               | <b>15</b> |
| 7.1. Pruebas del ProductoService . . . . .              | 15        |
| 7.2. Pruebas del CategoriaService . . . . .             | 16        |
| 7.3. Pruebas del UsuarioService . . . . .               | 16        |
| 7.4. Pruebas del JwtTokenUtil . . . . .                 | 16        |
| 7.5. Pruebas del modelo Producto . . . . .              | 16        |
| <b>8. Gestión de secretos</b>                           | <b>18</b> |
| 8.1. Secretos configurados . . . . .                    | 19        |
| <b>9. Registro de contenedores (Docker Hub)</b>         | <b>20</b> |
| 9.1. Estrategia de etiquetado . . . . .                 | 20        |
| 9.2. Dockerfiles (Multi-stage builds) . . . . .         | 21        |
| <b>10.Resultados obtenidos</b>                          | <b>24</b> |
| 10.1. Ejecución exitosa del pipeline completo . . . . . | 24        |
| 10.2. Detalle de cada job . . . . .                     | 25        |
| 10.3. Verificación del despliegue . . . . .             | 29        |
| 10.4. Notificación recibida . . . . .                   | 30        |

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>11. Análisis y conclusiones</b>                | <b>30</b> |
| 11.1. Beneficios del CI/CD implementado . . . . . | 30        |
| 11.2. Lecciones aprendidas . . . . .              | 31        |

# 1. Introducción

En este trabajo se implementa un pipeline de Integración Continua y Entrega Continua (CI/CD) automatizado para la aplicación **E-Converse**, una plataforma de comercio electrónico desarrollada con Spring Boot (backend), React con Vite (frontend) y MongoDB (base de datos).

El objetivo principal es automatizar el proceso completo de entrega de software: desde la compilación y ejecución de pruebas unitarias, pasando por la construcción y publicación de imágenes Docker en Docker Hub, hasta el despliegue en un servidor remoto mediante SSH. Adicionalmente, se implementa un sistema de notificaciones por correo electrónico para informar sobre el estado de cada etapa del pipeline.

La herramienta seleccionada para la implementación del pipeline es **GitHub Actions**, la cual permite definir flujos de trabajo (workflows) directamente en el repositorio de código fuente mediante archivos YAML.

## 2. Descripción general del sistema

### 2.1. Stack tecnológico

- **Backend:** Spring Boot 3.5.6 (Java 17), Spring Security con JWT, Spring Data MongoDB
- **Frontend:** React 19 con Vite 7, TailwindCSS 3.4, Zustand para estado global
- **Base de datos:** MongoDB Atlas (NoSQL)
- **Contenedores:** Docker con multi-stage builds
- **CI/CD:** GitHub Actions
- **Registro de imágenes:** Docker Hub

### 2.2. Arquitectura de la aplicación

E-Converse es una tienda virtual con gestión integral de usuarios, productos, carrito de compras, pasarela de pagos simulada y gestión de pedidos. La comunicación entre frontend y backend se realiza mediante una API REST protegida con JWT.

Imágenes/arquitectura\_general.png

Figura 1: Arquitectura general de E-Converse

### 3. Estructura del repositorio

El proyecto se organiza en dos directorios principales: **backend/** para la API REST con Spring Boot y **frontend-react/** para la SPA con React. El pipeline de CI/CD se define en `.github/workflows/ci-cd.yml`.

```
Proyecto-Spring-Boot-E-converse_vers2/
|-- .github/
|   |-- workflows/
|       |-- ci-cd.yml                # Pipeline CI/CD
|-- backend/
|   |-- Dockerfile                  # Multi-stage build
|   |-- pom.xml                     # Dependencias Maven
|   |-- src/main/java/              # Código fuente
|   |-- src/test/java/              # Pruebas unitarias
|-- frontend-react/
```

```
|    |-- Dockerfile          # Multi-stage build
|    |-- nginx.conf         # Config Nginx
|    |-- package.json       # Dependencias Node
|    |-- src/               # Código fuente React
|-- docker-compose.yml      # Compose local
|-- informe/               # Este informe
```



Figura 2: Estructura del repositorio en GitHub

## 4. Conceptos fundamentales de CI/CD

### 4.1. Integración Continua (CI)

La Integración Continua es la práctica de fusionar frecuentemente los cambios de código de todos los desarrolladores en una rama compartida. Cada fusión activa un proceso automatizado que compila el código y ejecuta pruebas para detectar errores tempranamente.

### 4.2. Entrega Continua (CD)

La Entrega Continua extiende la CI automatizando el despliegue del software en entornos de prueba o producción. El objetivo es que cada cambio que pasa las pruebas esté listo para ser desplegado en cualquier momento.

### 4.3. Conceptos de GitHub Actions

- **Workflow:** Proceso automatizado definido en un archivo YAML dentro de `.github/workflows/`. Se activa por eventos como `push` o `pull_request`.
- **Job:** Conjunto de pasos (steps) que se ejecutan en un mismo runner. Los jobs pueden ejecutarse en paralelo o secuencialmente usando dependencias.
- **Step:** Acción individual dentro de un job. Puede ser un comando shell o una acción predefinida.
- **Runner:** Máquina virtual donde se ejecutan los jobs. GitHub proporciona runners con Ubuntu, Windows y macOS.
- **Artifact:** Archivo generado durante la ejecución del workflow que puede ser descargado o utilizado por otros jobs.
- **Secret:** Variable cifrada almacenada en la configuración del repositorio para proteger credenciales sensibles.

## 5. Estrategia de Branching

Para este proyecto se utilizó una estrategia basada en **GitHub Flow**, creando una rama específica `ci-cd-pipeline` desde `main` para todo el desarrollo del pipeline. Esto permite:

- No alterar la rama principal durante el desarrollo.
- Activar el pipeline automáticamente ante cada push a la rama.
- Facilitar la revisión mediante Pull Requests antes de integrar a `main`.

```
# Crear la rama de trabajo
git checkout -b ci-cd-pipeline

# Trabajar en la rama
git add .
git commit -m "feat: implementar pipeline CI/CD"
git push origin ci-cd-pipeline
```



Imagenes/branching\_strategy.png

Figura 3: Estrategia de branching utilizada

## 6. Explicación del Pipeline CI/CD

El pipeline se compone de **4 jobs secuenciales** definidos en el archivo `.github/workflows/ci-cd.yml`.

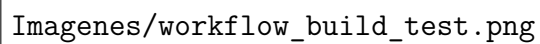
1. **Build & Test** — Compilar y ejecutar pruebas
2. **Docker Build & Push** — Construir y subir imágenes Docker
3. **Deploy** — Desplegar en servidor remoto via SSH
4. **Notify** — Notificar resultado por email

### 6.1. Job 1: Build & Test

Este job se encarga de compilar tanto el backend como el frontend y ejecutar las pruebas unitarias del backend.

**Pasos principales:**

1. Clonar el repositorio usando `actions/checkout@v4`
2. Configurar JDK 17 con `actions/setup-java@v4` y cache de Maven
3. Compilar el backend: `./mvnw -B clean compile`
4. Ejecutar pruebas unitarias: `./mvnw -B test`
5. Empaquetar JAR: `./mvnw -B package -DskipTests`
6. Subir artefacto JAR con `actions/upload-artifact@v4`
7. Configurar Node.js 20 e instalar dependencias del frontend
8. Ejecutar lint y build del frontend con Vite



Imagenes/workflow\_build\_test.png

Figura 4: Configuración del Job Build & Test en el workflow YAML

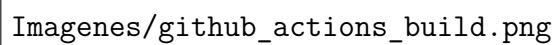


Figura 5: Ejecución del Job Build & Test en GitHub Actions

## 6.2. Job 2: Docker Build & Push

Una vez que el Job 1 finaliza exitosamente, este job construye las imágenes Docker del backend y frontend, y las sube a Docker Hub.

### Características:

- Login a Docker Hub usando secretos (`DOCKERHUB_TOKEN`)
- Docker Buildx para builds optimizados con cache
- Multi-stage builds: compilación separada del runtime
- Doble etiquetado: `latest` y SHA del commit

```
# Etiquetas generadas para cada imagen
lyoks/e-converse-backend:latest
lyoks/e-converse-backend:<sha-del-commit>
lyoks/e-converse-frontend:latest
lyoks/e-converse-frontend:<sha-del-commit>
```

Imagenes/workflow\_docker\_push.png

Figura 6: Configuración del Job Docker Build & Push



Imagenes/dockerhub\_images.png

Figura 7: Imágenes publicadas en Docker Hub

### 6.3. Job 3: Deploy

Este job despliega la aplicación en un servidor remoto (máquina virtual) mediante SSH. Utiliza la acción `appleboy/ssh-action@v1` para conectarse al servidor y ejecutar comandos Docker.

**Proceso de despliegue:**

1. Conectar al servidor vía SSH con credenciales almacenadas en secretos
2. Crear/actualizar el archivo `docker-compose.yml` en el servidor
3. Descargar las imágenes más recientes (`docker pull`)
4. Detener los contenedores anteriores (`docker compose down`)

5. Levantar los nuevos contenedores (`docker compose up -d`)
6. Verificar el estado de los contenedores



Figura 8: Configuración del Job Deploy vía SSH

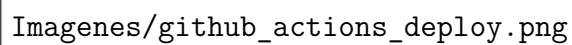


Figura 9: Ejecución del despliegue en GitHub Actions

#### 6.4. Job 4: Notificaciones

El último job envía una notificación por correo electrónico informando el resultado de cada etapa del pipeline. Se ejecuta **siempre** (con `if: always()`), independientemente de si los jobs anteriores fueron exitosos o fallaron.

**Información incluida en el correo:**

- Repositorio, rama y commit
- Estado de cada job (éxito/fallo)
- Mensaje del commit
- Enlace directo a la ejecución en GitHub Actions

- Nombres de las imágenes Docker generadas



Figura 10: Configuración del Job de notificación por email



Imagenes/email\_notification.png

Figura 11: Correo de notificación recibido

## 7. Pruebas unitarias implementadas

Se crearon pruebas unitarias para validar la lógica de negocio del backend usando **JUnit 5** y **Mockito**. Las pruebas mockean los repositorios de MongoDB para ejecutarse sin necesidad de una base de datos real.

### 7.1. Pruebas del ProductService

- Listar todos los productos
- Crear producto con y sin categoría
- Buscar producto por ID existente e inexistente
- Eliminar producto
- Verificación de precio y stock

## **7.2. Pruebas del CategoriaService**

- Listar todas las categorías
- Crear nueva categoría
- Buscar categoría por ID
- Eliminar categoría
- Actualizar categoría existente

## **7.3. Pruebas del UsuarioService**

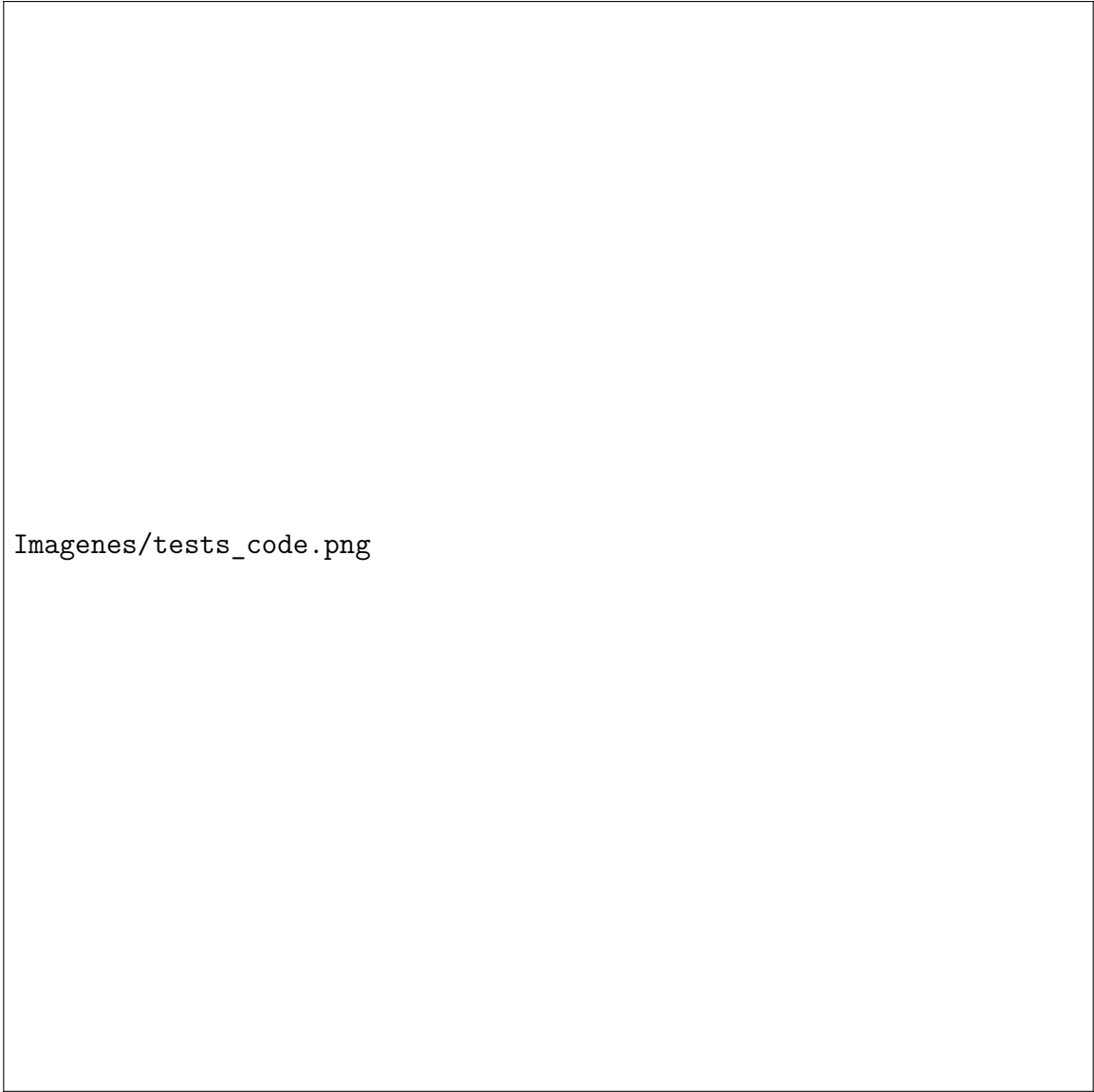
- Listar usuarios con roles
- Crear usuario con contraseña encriptada (BCrypt)
- Buscar usuario por ID
- Actualizar usuario sin re-encriptar contraseña
- Eliminar usuario

## **7.4. Pruebas del JwtTokenUtil**

- Generar token JWT válido (estructura de 3 partes)
- Extraer username del token
- Validar token con usuario correcto e incorrecto
- Manejo de tokens malformados e inválidos
- Tokens diferentes para diferentes usuarios

## **7.5. Pruebas del modelo Producto**

- Constructor vacío y constructor completo
- Getters y setters de todos los campos
- Manejo de precios con decimales
- Manejo de stock cero

A large rectangular box with a thin black border, intended to contain the unit test code example for ProductServiceTest.

`Imagenes/tests_code.png`

Figura 12: Ejemplo de código de pruebas unitarias (ProductoServiceTest)



Imagenes/tests\_results.png

Figura 13: Resultados de la ejecución de pruebas unitarias

## 8. Gestión de secretos

GitHub Actions provee un mecanismo seguro para almacenar credenciales sensibles mediante **Repository Secrets**. Los secretos se configuran en **Settings** → **Secrets and variables** → **Actions** del repositorio.

## 8.1. Secretos configurados

| Secreto                 | Descripción                                        |
|-------------------------|----------------------------------------------------|
| DOCKERHUB_TOKEN         | Token de acceso a Docker Hub para push de imágenes |
| SPRING_DATA_MONGODB_URI | URI de conexión a MongoDB Atlas                    |
| DEPLOY_HOST             | IP del servidor de despliegue                      |
| DEPLOY_USER             | Usuario SSH del servidor                           |
| DEPLOY_PASSWORD         | Contraseña SSH del servidor                        |
| MAIL_USERNAME           | Email remitente (Gmail) para notificaciones        |
| MAIL_PASSWORD           | Contraseña de aplicación de Gmail                  |
| NOTIFICATION_EMAIL      | Email destinatario de las notificaciones           |

Cuadro 1: Secretos configurados en GitHub Actions

### Características de seguridad:

- Los secretos se cifran antes de almacenarse en GitHub
- No se imprimen en los logs de ejecución (se enmascaran con \*\*\*)
- Solo están disponibles para los workflows del repositorio
- Se referencian con la sintaxis `${{ secrets.NOMBRE }}`



Figura 14: Secretos configurados en el repositorio de GitHub

## 9. Registro de contenedores (Docker Hub)

Las imágenes Docker se publican en **Docker Hub** bajo el usuario **lyoks**. Se utilizan dos repositorios de imágenes:

- **lyoks/e-converse-backend** — Imagen del backend (Spring Boot + JRE 17)
- **lyoks/e-converse-frontend** — Imagen del frontend (Nginx + archivos estáticos)

### 9.1. Estrategia de etiquetado

Cada build genera dos tags:

- **latest** — Siempre apunta a la versión más reciente
- **<sha>** — Tag inmutable con el SHA del commit, permite trazabilidad

## 9.2. Dockerfiles (Multi-stage builds)

**Backend:** Se utiliza un build en dos etapas: la primera compila el proyecto con Maven y la segunda ejecuta el JAR en un JRE mínimo.

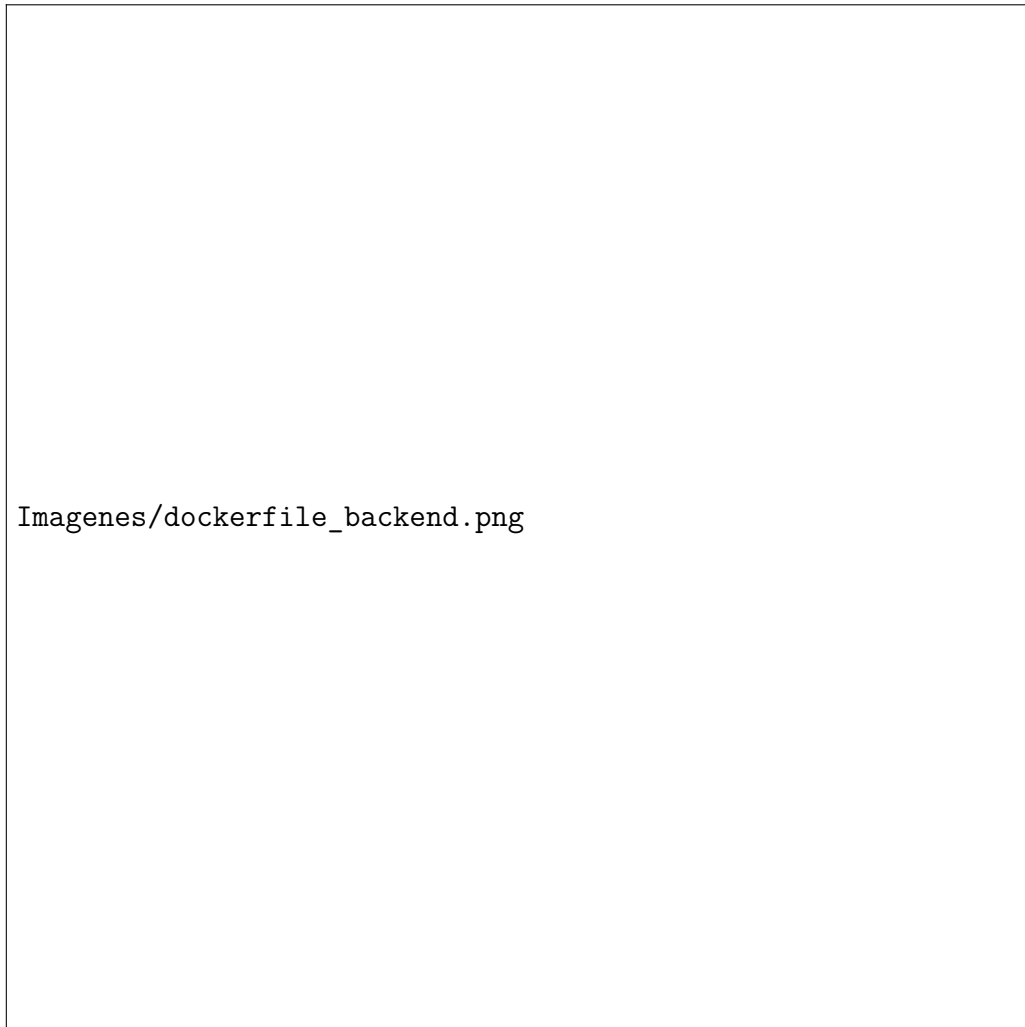


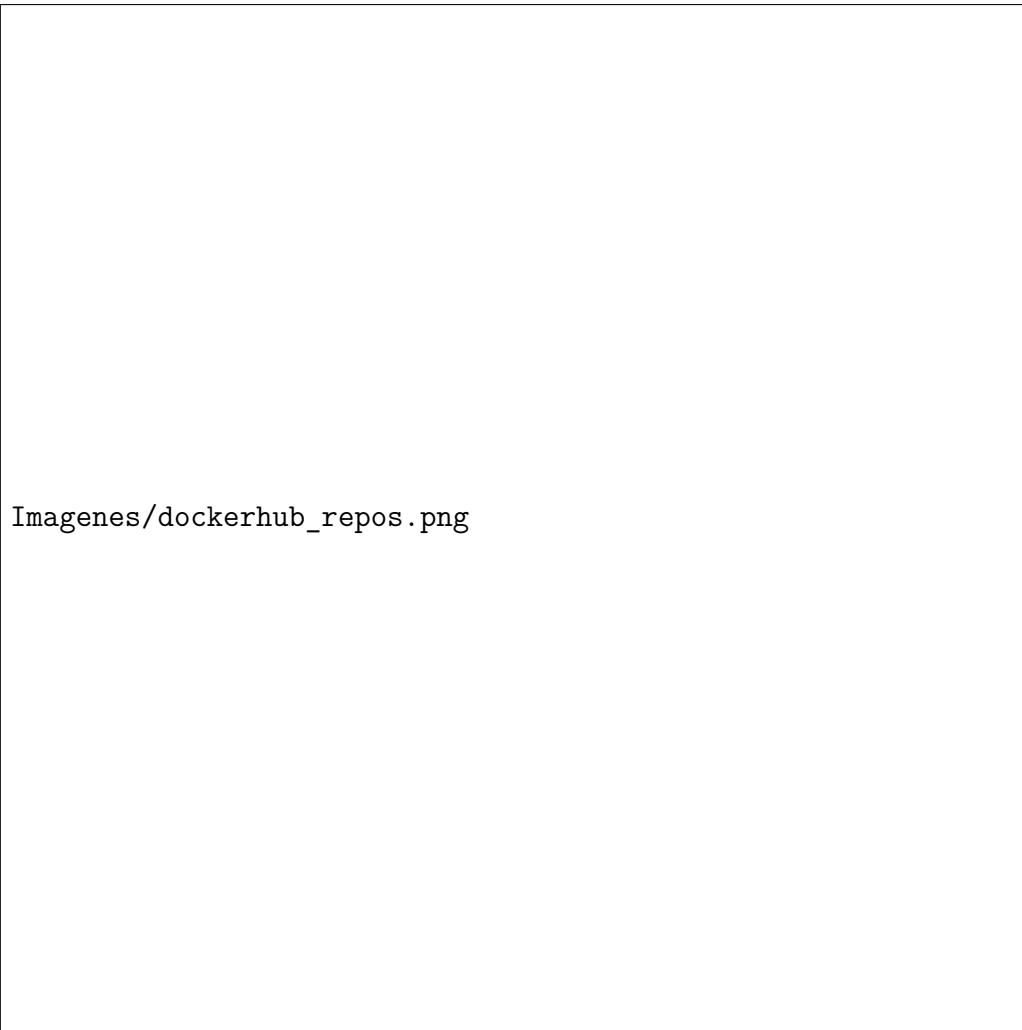
Figura 15: Dockerfile del Backend (multi-stage build)

**Frontend:** Se compila el proyecto con Node.js/Vite y se sirve con Nginx Alpine.



Imagenes/dockerfile\_frontend.png

Figura 16: Dockerfile del Frontend (multi-stage build)



Imagenes/dockerhub\_repos.png

Figura 17: Repositorios de imágenes en Docker Hub

## 10. Resultados obtenidos

### 10.1. Ejecución exitosa del pipeline completo



Figura 18: Pipeline completo ejecutado exitosamente en GitHub Actions

## 10.2. Detalle de cada job

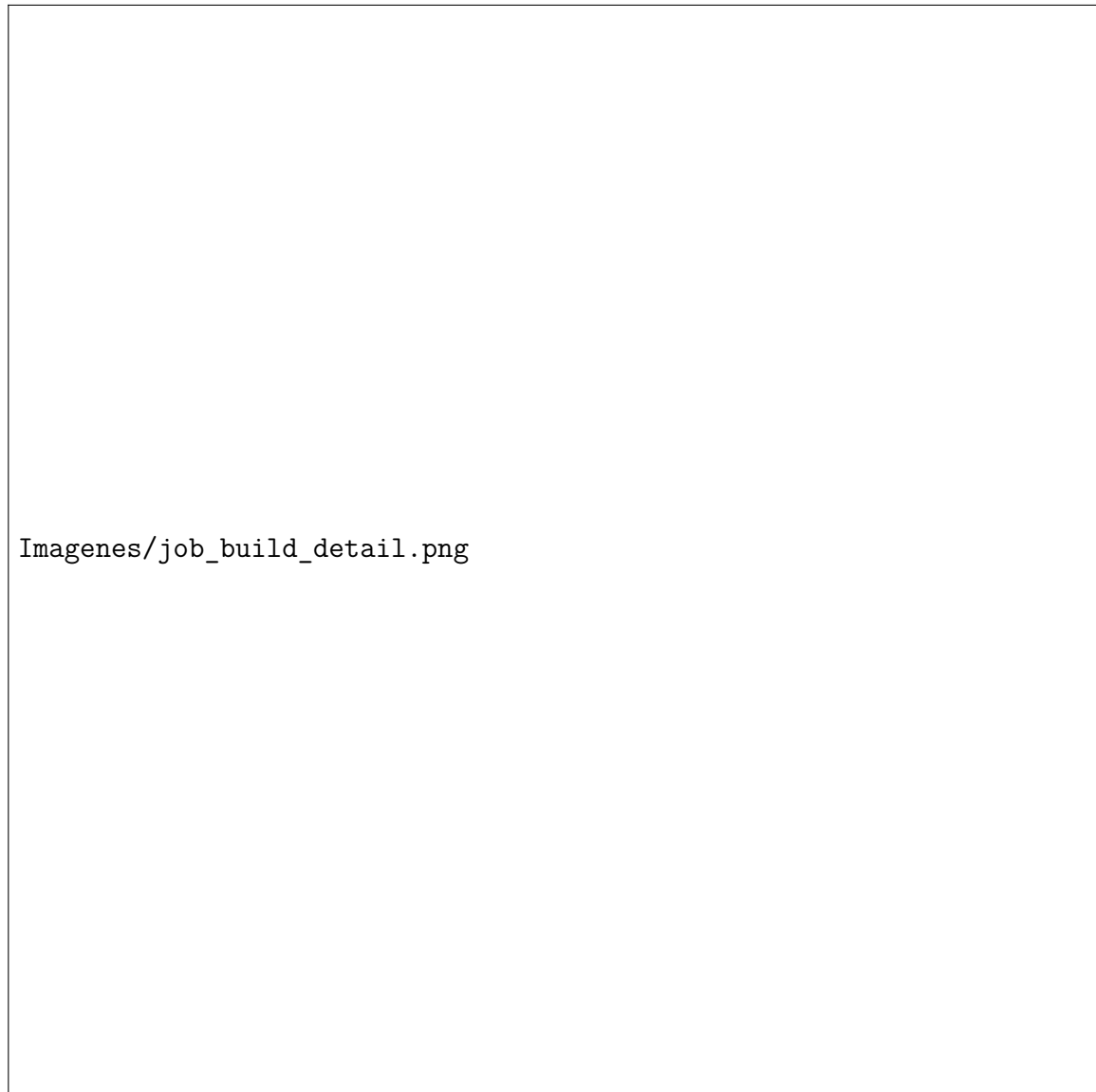
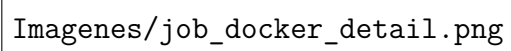
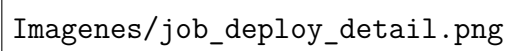


Figura 19: Detalle del Job Build & Test

The image area is mostly blank, with the text 'Imagenes/job\_docker\_detail.png' located on the left side. This likely represents a screenshot of a Docker build and push process, which is the subject of the figure caption.

Imagenes/job\_docker\_detail.png

Figura 20: Detalle del Job Docker Build & Push

A large rectangular box with a thin black border, intended for a screenshot or diagram. It contains the text 'Imagenes/job\_deploy\_detail.png' in the lower-left corner.

Imagenes/job\_deploy\_detail.png

Figura 21: Detalle del Job Deploy

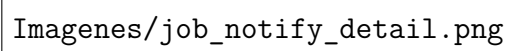
The image area is a large, empty rectangular box with a thin black border. Inside the box, on the left side, is the text 'Imagenes/job\_notify\_detail.png' in a monospaced font. This text likely serves as a placeholder for a screenshot or diagram related to the 'Job Notify' feature.

Figura 22: Detalle del Job Notify

### 10.3. Verificación del despliegue

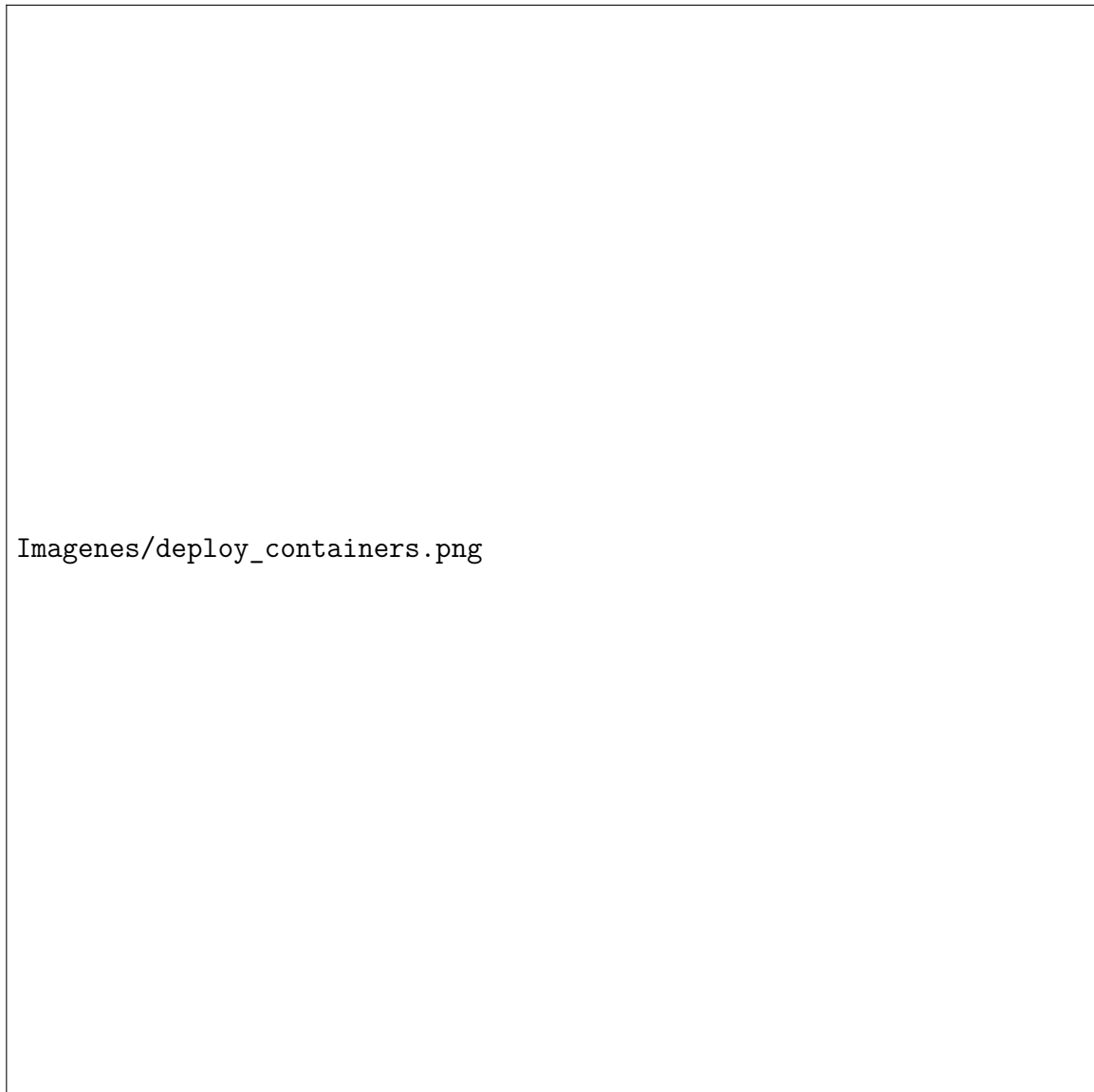


Figura 23: Contenedores ejecutándose en el servidor de despliegue

## 10.4. Notificación recibida

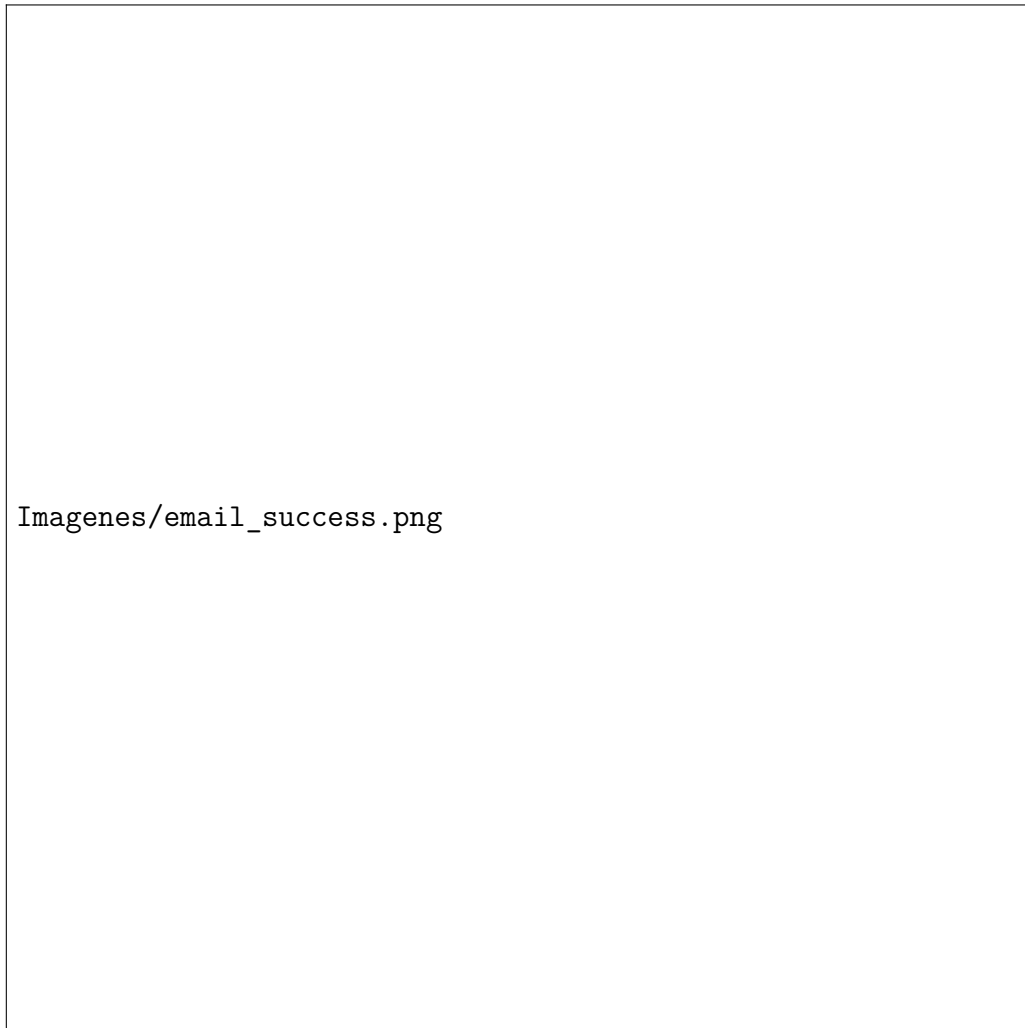


Figura 24: Correo de notificación de éxito recibido

## 11. Análisis y conclusiones

### 11.1. Beneficios del CI/CD implementado

- **Automatización completa:** Desde el commit hasta el despliegue, sin intervención manual.
- **Detección temprana de errores:** Las pruebas unitarias se ejecutan en cada push, evitando que código defectuoso llegue a producción.
- **Trazabilidad:** El etiquetado de imágenes con SHA permite rastrear exactamente qué commit está desplegado.
- **Seguridad:** Los secretos se gestionan de forma cifrada mediante GitHub Secrets.

- **Visibilidad:** Las notificaciones por email mantienen al equipo informado del estado del pipeline.
- **Reproducibilidad:** El uso de Docker garantiza que la aplicación se ejecute de forma idéntica en cualquier entorno.

## 11.2. Lecciones aprendidas

- La configuración de secretos en GitHub Actions es fundamental para proteger credenciales sensibles.
- Los multi-stage builds de Docker reducen significativamente el tamaño de las imágenes finales.
- La estrategia de branching (GitHub Flow) facilita la integración del pipeline con el flujo de desarrollo.
- Las notificaciones por email permiten reaccionar rápidamente ante fallos en el pipeline.
- El despliegue mediante SSH permite actualizar servidores remotos de forma automatizada y segura.